

ECMM409 - Nature-Inspired Computation

Group Report

Group 11

Student Numbers:

740089126, 740099118, 740002131,
740090003, 740079735, 740045148

Abstract

This paper explores how six different Nature Inspired algorithms solve the Travelling Thief Problem (TTP). Algorithms such as GA with Tabu Search, PSO, ACO and NSGA-II were employed. Among these, NSGA-II had the best performance, as it achieved great performance and superior Pareto front coverage, effectively balancing travel time with profit. These findings show and highlight the potential bi-objective optimisation for solving interdependent problems.

1 Introduction

The Travelling Thief Problem (TTP) is a bi-objective optimisation problem that integrates the Travelling Salesman Problem (TSP) and the Knapsack Problem. This study outlines our group's approach to addressing the TTP. Our methodology involved each group member conducting a literature review, identifying and implementing an algorithm to solve the TTP, and engaging in a collaborative analysis to determine the most effective approach. This report begins by discussing the selected algorithms and the rationale behind their selection. We then detail the steps taken to ensure a fair experimental setup and comparison of the algorithms. Following this, we present our findings and identify the winning algorithm. Finally, we conclude with a discussion of potential directions for future work.

2 Literature Review

Six distinct approaches were employed to address the Travelling Thief Problem (TTP), each exploring different strategies to balance the interdependence of the Travelling Salesman Problem (TSP) and the Knapsack Problem (KP).

The first approach utilised a hybrid genetic algorithm combined with Tabu Search, aiming to optimise the TSP and KP components simultaneously. This method presented a novel approach to TTP by integrating these sub-problems for joint optimisation [1].

The second approach, MA2B, employed a memetic algorithm, an extension of a Genetic Algorithm incorporating Restrictive Local Search. This integrated two-opt and bit-flip local search techniques [5].

The third approach implemented Ant Colony Optimisation (ACO), which uses a distribution of tours rather than just a single tour, where the ants are responsible for generating tours. The packing plan is then generated with a greedy heuristic for the tours [8].

The fourth approach applied NSGA-II, a multi-objective evolutionary algorithm, to solve the TTP. The NSGA-II algorithm applies non-dominated sorting and crowding-distance techniques to generate diverse, high-quality solutions. [3].

The fifth approach utilised Particle Swarm Optimisation (PSO), which works by iteratively moving a population of candidate solutions (particles) through the search space, guided by their own best-known positions and the swarm's global best position. [9].

The sixth approach was a second memetic algorithm, integrating Tabu search with a genetic algorithm. This improved the performance by not revisiting already explored solutions, which promoted diversity by exploring unexplored regions of the objective space. [2].

3 Experimental Design

This study employed a rigorous experimental design to ensure the comparability and reliability of our group's selected algorithms to solve the Travelling Thief Problem (TTP). The design included consistency in the evaluation framework, objec-

tive functions, stopping criteria, solution filtering methods, and test instances.

3.1 Objective Functions

The primary objectives of this study were to minimise the total time taken for the tour and maximise the profit obtained from collected items. As a team, we decided not to use the renting ratio to combine both objectives to measure fitness. Instead, we employed the tour time and knapsack profit as the evaluation metrics.

3.2 Stopping Criteria

A fixed number of total fitness evaluations were used as the stopping criteria for all algorithms. The numbers decided were 200,000 total fitness evaluations for the a280 test files, 50,000 evaluations for the FNL test files, and 2600 total fitness evaluations for the PLA files. This decision ensured fairness when comparing different algorithms and implementations. We did not use time-based stopping criteria, as we all implemented our algorithms on our own devices, and these hardware discrepancies would have led to unfair comparisons.

3.3 Filtering Solutions

We applied a Non-Dominated Sorting filtering process to filter optimal solutions. We used Non-Dominated Sorting to identify the solutions on the approximated Pareto front. Non-dominated sorting ensures that all selected solutions are optimal in at least one objective. [7]

3.4 Test Instances

As specified in the competition, we evaluated all algorithms against the nine benchmark test instances of the competition. These instances, being diverse in scale and complexity, provided insights into the performance of our algorithms against small, medium and large datasets.

4 Results

Table 1 displays the normalised hypervolumes of the algorithms for the a280-n2790 test problem - MA2B, Genetic Algorithm, NSGA-II, ACO, MATL, and PSO, in optimising the Travelling Thief Problem (TTP) by balancing travel time and profit. NSGA-II achieved the highest normalised hypervolume of 0.99, indicating superior Pareto front coverage, followed closely by PSO (0.9795)

and ACO (0.9167), while MATL and MA2B show relatively low performance.

Algorithm	Hypervolume
NSGA-II	0.9900
PSO	0.9795
ACO	0.9167
Genetic Algorithm	0.8650
MA2B	0.6155
MATL	0.5283

Table 1: Normalised hypervolume values for the selected algorithms for the a280-n2790 test.

Figure 1 shows the performance of all the algorithms. The Pareto front in the graph highlights the trade-offs achieved by each algorithm. NSGA-II (green) and PSO (brown) demonstrate superior convergence with solutions distributed near the Pareto front, as indicated by their high normalised hypervolume values of 0.9900 and 0.9795, respectively. Genetic Algorithm (orange) and ACO (red) also perform well, achieving normalised hypervolumes of 0.8650 and 0.9167, respectively, but show less diversity in the solution space. MA2B (blue) achieves a normalised hypervolume of 0.6155, suggesting moderate performance despite displaying good diversity. MATL (purple) achieves a low hypervolume of 0.5283, indicating limited quality and diversity of solutions.

Figures 2-3 highlight the trade-off between profit and travel time for different test instances for the NSGA-II algorithm. The graph exhibits a clear Pareto front where increasing travel time results in a decrease in negative profit (indicating higher profit). The points suggest a balanced and consistent behaviour of NSGA-II in effectively solving multi-objective optimisation tasks for the problem.

NSGA-II outperforms other algorithms in terms of normalised hypervolume and good solutions, reflecting its ability to balance exploration and exploitation effectively. PSO also achieves competitive results compared to NSGA-II; on the other hand, other algorithms' subpar performance highlights their struggle to generate diverse and high-quality solutions. Overall, NSGA-II and PSO provide the best trade-offs between travel time and profit, with NSGA-II leading in terms of hypervolume performance. In the following sub-section, we will dive deeper into the implementation of the winning NSGA-II algorithm.

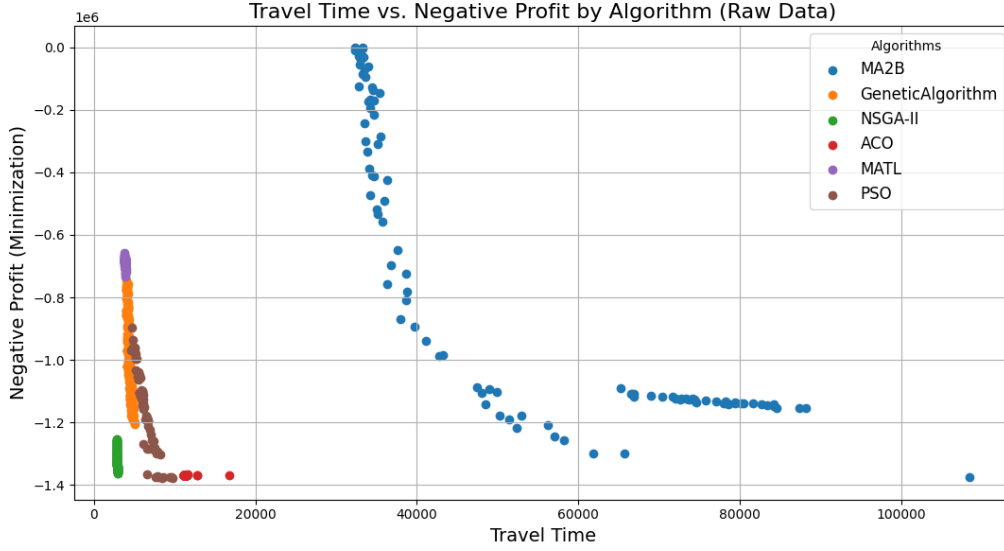


Figure 1: Performance comparison of selected algorithms highlighting trade-offs between travel time and profit.

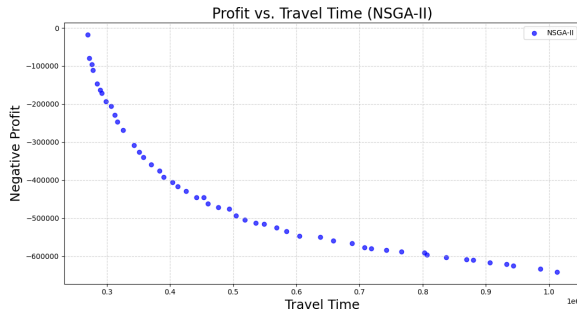


Figure 2: fnl4461-n4460

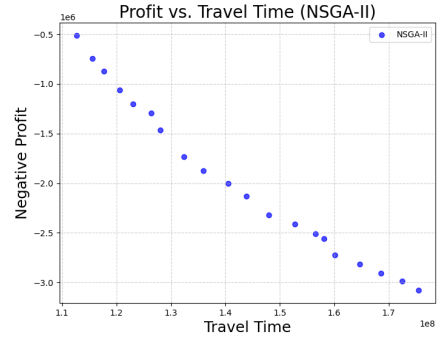


Figure 3: pla33810-n33809

4.1 Winning Algorithm

The winning algorithm utilised a combination of the Non-dominated Sorting Genetic Algorithm II (NSGA-II) and Ant Colony Optimisation (ACO), leveraging the strengths of both methods to achieve competitive results in solving the Travelling Thief Problem (TTP).

4.1.1 Representations

The solution representation for the algorithm was as follows:

- **TSP Representation:** A permutation array where each index represents an ordered visit to nodes. The order of numbers in the array determines the travel path.
- **Knapsack Representation:** A binary array where each index corresponds to an item. A value of "0" indicates that the item is not

picked, while "1" signifies that the item is picked in the knapsack. The profit and weight of picked items are added to the knapsack.

4.1.2 Generating Initial Population

The initial population for the algorithm was generated using the following methods:

1. **Tour Initialisation:** Ant Colony Optimisation (ACO) was employed to generate initial tours. ACO is inspired by the foraging behaviour of ants, where pheromones guide ants to shorter paths. [4] In this case, the heuristic component used was $\frac{1}{\text{distance}}$, favouring closer nodes. The probability of selecting the next node increases as the pheromone and heuristic value increase. Key parameters for ACO included:

- $\alpha = 1$: Weight assigned to the pheromone importance.

- $\beta = 5$: Weight assigned to the heuristic value. A higher value of β ensures quicker convergence of the ants to feasible solutions, providing high-quality initial solutions for the evolutionary process.

2. **Knapsack Initialisation:** The knapsack's picking plan in the initial population was generated using a greedy search heuristic, guided by a profit-to-weight ratio with a randomness factor to encourage exploration. Items were prioritised based on their $\frac{\text{profit}}{\text{weight}}$ ratio, and each item's inclusion was constrained by the total knapsack capacity.

To ensure diversity in the picking plans, every generated parent solution was assigned a unique capacity constraint, randomly selected between zero and the total knapsack capacity, with a slightly higher probability of selecting weights closer to 0 and the total knapsack capacity for the given test problem. This strategy allowed lighter knapsacks to promote solutions that prioritised minimising the time taken by the tour over the total profit of the knapsack.

4.1.3 NSGA-II and the Evolutionary Process

NSGA-II is a fast, non-dominated sorting multi-objective evolutionary algorithm. It employs elitism to preserve the best solutions and a crowding distance metric to maintain diversity within the population.

Evolutionary Parameters

- **a280 Test Instances:** The algorithm used a population size of 200, generating 200 offspring per generation, over 1,000 generations.
- **FNL Test Instances:** A smaller parent population of 5 was used, with an additional 45 offspring generated during each evolutionary cycle, over 1,000 generations.
- **PLA Test Instances:** The parent population was set to 2, with 24 offspring generated during the evolutionary process, continuing for 100 generations.

Genetic Operators

- **Crossover**

- **TSP Tour:** Order crossover was applied to generate offspring. This method preserved the order of nodes from parent tours while ensuring valid permutations.
- **Knapsack Picking Plan:** Single-Point crossover was employed here. A segment N (1 to knapsack length) from Parent 1 was combined with the remaining portion from Parent 2. If the resulting picking plan violated the capacity constraint, the offspring reverted to Parent 1's picking plan.

• Mutation

- **Knapsack Picking Plan:** Bit-flip mutation was applied to each offspring, inverting every bit in the picking plan. If the mutated solution exceeded the capacity constraint, the mutation was rejected, and the offspring reverted to its original state.

Selection and Diversity Mechanisms

- **Selection:** Tournament selection was used, with two tournaments involving two individuals each. The winners were chosen for the next generation.
- **Elitism:** NSGA-II combined the parent population and offspring, retaining only the top-ranked solutions based on Pareto dominance. The population size was kept constant through generations.
- **Crowding Distance and Diversity Preservation:** Non-dominated sorting was used to rank solutions. Within the last required rank, solutions were prioritised based on their crowding distance. This metric favoured solutions farther apart in the objective space, ensuring diversity along the Pareto front.

5 Conclusion

Among all the algorithms that were studied, NSGA-II had the best performance across all the test instances and the project was an overall success. If given more time, we would explore more famous MOEAs such as SMS-EMOA and MOEA/D.

References

- [1] Saad Alharbi. “A Hybrid Genetic Algorithm with Tabu Search for Optimization of the Traveling Thief Problem”. In: *International Journal of Advanced Computer Science and Applications* 9 (Jan. 2018). DOI: 10.14569/IJACSA.2018.091138.
- [2] Mohammad Reza Bonyadi, Zbigniew Michalewicz, and Luigi Barone. “The travelling thief problem: The first step in the transition from theoretical problems to realistic problems”. In: *2013 IEEE Congress on Evolutionary Computation*. 2013, pp. 1037–1044. DOI: 10.1109/CEC.2013.6557681.
- [3] K. Deb et al. “A fast and elitist multiobjective genetic algorithm: NSGA-II”. In: *IEEE Transactions on Evolutionary Computation* 6.2 (2002), pp. 182–197. DOI: 10.1109/4235.996017.
- [4] Marco Dorigo, Mauro Birattari, and Thomas Stutzle. “Ant colony optimization”. In: *IEEE Computational Intelligence Magazine* 1.4 (2006), pp. 28–39. DOI: 10.1109/MCI.2006.329691.
- [5] Mohamed El Yafrani and Belaid Ahiod. “Population-based vs. Single-solution Heuristics for the Travelling Thief Problem”. In: *Proceedings of the Genetic and Evolutionary Computation Conference 2016*. GECCO ’16. Denver, Colorado, USA: Association for Computing Machinery, 2016, pp. 317–324. ISBN: 9781450342063. DOI: 10.1145/2908812.2908847. URL: <https://doi.org/10.1145/2908812.2908847>.
- [6] Ke Shang et al. “A Survey on the Hypervolume Indicator in Evolutionary Multiobjective Optimization”. In: *IEEE Transactions on Evolutionary Computation* 25.1 (2021), pp. 1–20. DOI: 10.1109/TEVC.2020.3013290.
- [7] N. Srinivas and Kalyanmoy Deb. “Multiobjective Optimization Using Nondominated Sorting in Genetic Algorithms”. In: *Evolutionary Computation* 2.3 (1994), pp. 221–248. DOI: 10.1162/evco.1994.2.3.221.
- [8] Markus Wagner. “Stealing Items More Efficiently with Ants: A Swarm Intelligence Approach to the Travelling Thief Problem”. In: Sept. 2016, pp. 273–281. ISBN: 978-3-319-44426-0. DOI: 10.1007/978-3-319-44427-7_25.
- [9] Jicheng Yao et al. “Research on hybrid strategy Particle Swarm Optimization algorithm and its applications”. In: *Scientific Reports* 14.1 (2024), p. 24928. DOI: 10.1038/s41598-024-76010-y. URL: <https://doi.org/10.1038/s41598-024-76010-y>.